# MlasQgemmKernel VL=16 多平台向量实现对比与RVV扩展指令建议

# 概述

分析目标: `MlasQgemmKernelRvv512Impl` — INT8量化矩阵乘法核心kernel（uint8×uint8uint32） 基准实现: RVV VLEN=512, VL=16 (uint32), VL8 (uint8), LMUL=1 分析平台: x86 AVX512_VNNI, ARM NEON UDOT/SMMLA, LoongArch LSX, Power VSX MMA, S390X NNPI, WASM SIMD **BBV**数据: 未提供，收益为理论估算（基于MLAS profiling热点占比约73.97%）

# 指令方案汇总

| 优先级 | 扩展指令 | 来源平台 | **BB**内收益 | 实现难度 | **RVV**现状 |
|---|---|---|---|---|---|
| P0 | vwmaccu8x2.vv | x86 VNNI / ARM UDOT | K循环BB内减少50%（4 K元素） | 中 | RVV有vwmaccu但仅uint8uint16单步扩展 |
| P1 | vmatmul.rs.vv | ARM SMMLA | 矩阵块BB内减少75%（32 MACs） | 高 | RVV无矩阵乘指令 |
| P2 | vwmaccsu8x2.vv | x86 VNNI | 符号混合场景简化 | 低 | 需手动XOR 0x80 |
| P3 | vwmaccu8x2s.vv | x86 VNNI | 饱和累加场景 | 低 | RVV无饱和版本 |

收益计算方式（无BBV数据，仅BB范围内估算）：

- BB内收益 = (原BB指令数 - 扩展后BB指令数) / 原BB指令数 × 100%
- 整体收益需BBV profiling数据支持，建议通过 `./tools/profile_to_dfg.sh` 获取

# 基准RVV实现分析

## RVV实现结构 (VLEN=512, VL=16)

```
// rvv_qgemm_kernel_vl16.inl 核心K循环结构

for (size_t pk = 0; pk < PackedCountK; pk++) {
    // K元素0: a0 * B[0..15]
    vuint8mf4_t vb0 = __riscv_vle8_v_u8mf4(b, vl8); // 1条加载
    vuint16mf2_t vp0 = __riscv_vwmulu_vx_u16mf2(vb0, a0, vl8); // 1条widening mul
    vacc = __riscv_vwaddu_wv_u32m1(vacc, vp0, vl); // 1条widening add

    // K元素1-3: 同理，各3条指令
    ...
    a += 4;
    b += 64; // 4 K元素 × 16 columns = 64 bytes
}
```

## RVV指令计数分析

| 操作阶段 | 指令数 | 元素数 | 说明 |
|---|---|---|---|
| 单K元素MAC | 3 | 16列 | vle8 + vwmulu + vwaddu |
| 4 K元素迭代 | 12 | 16列×4K | 4×(加载+widening mul+widening add) |
| N循环单块 | 12×PackedCountK + 6 | 16列 | 主循环 + 累加器合并 |

关键观察：

- RVV INT8 MAC需要**2**条运算指令（widening multiply + widening add）
- 无专用INT8×INT8INT32融合指令
- widening操作从uint8uint16uint32，两级扩展
- PackedK=4，每迭代处理4个K元素

# 各平台对比分析

## 1. x86 AVX512_VNNI

核心特点：

- ZMM/YMM/XMM寄存器可选，512/256/128位向量宽度
- **VNNI**核心能力：单指令完成4×(uint8×int8) + int32累加
- ZMM宽度：64个int8 MACs/指令
- 多种符号组合：vpdpbusd (U×S)、vpdpbusds (U×S饱和)、vpdpbssd (S×S)、vpdpbssds (S×S饱和)

高价值指令：

| 指令 | 功能 | MACs | RVV现状 |
|---|---|---|---|
| vpdpbusd | 4×uint8×int8 + int32累加（非饱和） | 64/ZMM | RVV需2条：vwmulu_vx + vwaddu_wv（注：vwmulu为无符号乘法，uint8×int8需先XOR 0x80处理符号） |
| vpdpbusds | uint8×int8 饱和版本 | 64/ZMM | RVV无对应，需手动溢出检测 |
| vpdpbssd | int8×int8 非饱和 | 64/ZMM | RVV需符号扩展 |
| vpdpbssds | int8×int8 饱和版本 | 64/ZMM | RVV无对应，需手动溢出检测 |

指令语义：

```
VPDPBUSD dst, src1, src2 // 每个32位lane内执行:
  for i in 0..15 (zmm lanes):
    temp = 0
    for j in 0..3 (byte positions):
      temp += ZeroExtend(src1.byte[i][j]) * SignExtend(src2.byte[i][j])
    dst.dword[i] += temp
```

收益分析（归一化到**VLEN=512**，**16 outputs**，**4 K**元素）：

| 平台 | 每K元素指令数 | 4 K元素总指令 | 相对效率 |
|---|---|---|---|
| AVX512_VNNI | 1 (`vpdpbusd`) | 4 | 100%（基准） |
| AVX2无VNNI | 3 (`vpmaddubsw`+`vpmaddwd`+`vpaddd`) | 12 | 33% |
| RVV | 2 (`vwmulu_vx`+`vwaddu_wv`) + 1 load | 12（含加载） | 50% |

**RVV**差距：

1. 指令融合缺失：VNNI单指令完成MAC，RVV需分开执行widening multiply + widening add
2. 符号处理复杂：VNNI内置U×S/S×S组合，RVV需手动XOR 0x80处理
3. 饱和累加缺失：RVV无饱和版本，大累加值需额外溢出检测

建议扩展：

- `vwmaccu8x2.vv` — uint8×uint8uint32 双步扩展MAC（单指令，-50%指令数）
- `vwmaccsu8x2.vv` — int8×uint8 双步扩展MAC（符号混合）
- `vwmaccu8x2s.vv` — 饱和累加版本

## 2. ARM NEON UDOT/SMMLA

核心特点：

- ARMv8.4-a UDOT扩展：128-bit固定向量，dot product指令
- ARMv8.6-a I8MM扩展：**SMMLA**矩阵乘指令，单指令2×8 × 8×2矩阵乘
- 32个128-bit SIMD寄存器（v0-v31）
- 专门优化的interleaved数据打包格式

高价值指令：

| 指令 | 功能 | MACs/指令 | RVV现状 |
|---|---|---|---|
| `vudot.u8` | uint8×uint8uint32 dot product | 4 | RVV需2条/4 MACs |
| `vsdot.s8` | signed版dot product | 4 | 同上 |
| `smmla` | **2×8 × 8×2**矩阵乘（32 MACs） | **32** | RVV需4条指令（见下文） |
| `ummla` | unsigned矩阵乘 | 32 | 同上 |

**SMMLA**语义详解：

```
SMMLA Vd.4S, Vn.16B, Vm.16B
// Vn: 2 rows × 8 cols int8 (16 bytes)
// Vm: 8 rows × 2 cols int8 (16 bytes, column-packed)
// Vd: 2×2 int32 accumulator
// 总MACs = 4 outputs × 8 products = 32 MACs
```

收益分析（归一化到**VLEN=512**）：

| 操作类型 | ARM NEON | RVV | 效率差距 |
|---|---|---|---|
| 单次dot product (4 MACs) | 1条UDOT | 2条vwmulu+vwaddu | **2×** |

| 操作类型 | ARM NEON | RVV | 效率差距 |
|---|---|---|---|
| 32 MACs（等效1条SMMLA） | 1条SMMLA | 4条（2对vwmulu+vwmacc） | **4×** |

关键差距：

1. 无矩阵乘指令：SMMLA单指令32 MACs，RVV需4条指令完成相同工作量（每条vwmulu+vwmacc处理8个元素×2=16 MACs，两条对处理32 MACs）
2. **indexed dot product**缺失：ARM UDOT可用lane索引提取元素，RVV需显式slide
3. 数据重排开销：SMMLA interleaved packing vs RVV sequential layout

建议扩展：

- `vmatmul.rs.vv` — 2×8 × 8×2矩阵乘（参考SMMLA）
- `vdot.u8.vv` — 向量dot product
- `vwmaccu8x2.vv` — uint8 MAC直接累加到int32

## 3. LoongArch LSX

核心特点：

- 128-bit向量寄存器，8个int16元素/向量
- 专用**widening**乘加指令：`vmaddwev_w_h`（偶数位）、`vmaddwod_w_h`（奇数位）
- 元素分离处理：偶/奇两组并行处理，自动扩展到int32累加
- PackedK=2（每迭代2个K元素）

高价值指令：

| 指令 | 功能 | RVV现状 |
|---|---|---|
| `vmaddwev_w_h` | 偶数位widening MAC（int16×int16int32） | 无对应，需vwmulu+vwaddu |
| `vmaddwod_w_h` | 奇数位widening MAC | 无对应 |
| `vilvl_b` | bytehalf-word零扩展 | 有vzext.vf8 |

收益分析（归一化到**VLEN=512**，**64**元素）：

| 操作 | LSX指令数 | RVV指令数 | 备注 |
|---|---|---|---|
| 单MAC | 2 (vmaddwev+vmaddwod) | 2 (vwmulu+vwaddu) | 相同 |
| 8元素LSX向量 | 4+2 (归约) | 4+3 (归约) | LSX归约更紧凑 |

关键差异：

- **LSX**乘加融合与元素分离：`vmadd*`单指令完成乘加，但需偶/奇分离（相邻元素分布到不同指令）
- **RVV**分步实现与连续处理：widening multiply + widening add两步，但可连续处理所有元素
- 指令数权衡：LSX用2条指令（偶/奇分离）处理完整向量，RVV用2条指令（乘/加分离）处理完整向量，指令数相同但策略不同
- LSX PackedK=2 vs RVV PackedK=4

建议扩展：

- `vmaclu.vv` — widening MAC融合指令（偶数位）

- `vmacl2.vv` — 同时处理偶/奇位

## 4. Power VSX (POWER10) MMA

核心特点：

- 专用累加器寄存器：独立512位AR寄存器文件（AR0-AR7），与VSR分离
- **MMA**指令架构：外积（outer product）操作，单指令完成矩阵块乘累加
- **INT8**矩阵指令：`XVI8GER4PP`执行4×4 INT8外积累加到累加器
- 多精度支持：INT8/FP16/BF16/FP32

高价值指令：

| 指令 | 功能 | MACs | RVV现状 |
|---|---|---|---|
| `XVI8GER4PP` | INT8 4×4外积累加（4组4×4=64 MACs） | 64 | RVV需16条指令 |
| `XVI8GER4` | INT8 4×4外积（非累加） | 64 | 同上 |
| Accumulator Load/Store | 512位累加器操作 | — | 无专用累加器 |

**XVI8GER4PP**语义：

```
XVI8GER4PP ARd, VSn, VTm
// VSn: 16个INT8元素
// VTm: 16个INT8元素
// ARd: 4×4 INT32累加器（16个INT32）
// 外积操作: ARd[i][j] += VSn[i] * VTm[j]
// 总MACs = 16 × 4 = 64（每个输出4个乘积）
```

收益分析（**VLEN=512**）：

| 平台 | **4×4**块指令数 | **MACs/**指令 | 效率 |
|---|---|---|---|
| POWER10 MMA | 1 (`XVI8GER4PP`) | 64 | 16× RVV |
| RVV | 16 (vwmulu×8 + vwaddu×8) | 4 | 基准 |

建议扩展：

- `vmatmul.vv` — 4×4矩阵块乘法
- 专用累加器寄存器架构
- BF16支持

## 5. S390X NNPI

核心特点：

- IBM z16集成独立**AI**加速器（非通用向量ISA）
- Vector Facility仅128位通用SIMD，无**INT8**专用**MAC**指令
- 架构差异：专用加速器 vs 通用ISA扩展

高价值指令：

| 指令 | 功能 | RVV现状 |
|---|---|---|
| AI Accelerator INT8 | INT8矩阵乘/卷积 | 独立硬件，无ISA对应 |
| Vector Facility通用 | 128位SIMD | 无INT8专用MAC |

关键发现：

- S390X采用独立加速器方案，不属于通用ISA
- 无法直接对比ISA扩展
- 建议RVV采用通用ISA扩展（参考VNNI/UDOT而非S390X方案）

## 6. WASM SIMD

核心特点：

- 128-bit v128寄存器，16个int8/uint8
- `i32x4.dot_i16x8_s`：int16×int16点积int32（每条产生2个int32结果）
- 局限性：INT8需先扩展到INT16再做点积，无法直接INT8INT32

高价值指令：

| 指令 | 功能 | RVV现状 |
|---|---|---|
| `i32x4.dot_i16x8_s` | int16×int16int32 dot product | RVV有vwmacc类似能力 |

收益分析（归一化到**VLEN=512**）：

| 平台 | INT8INT32指令序列 | 总指令 | 相对效率 |
|---|---|---|---|
| WASM | dot_i16×2 + extend×2 | 4 | 0.5× RVV |
| RVV | vwmulu + vwaddu | 2 | 基准 |

关键发现：

- WASM SIMD INT8能力不及**RVV**
- 点积仅到int16，需额外扩展（+50%指令）
- RVV在此场景已具备优势，无需向WASM学习

# RVV扩展指令建议详细说明

## [P0] vwmaccu8x2.vv / vwmaccu8x2.vx

指令定义：

```
vwmaccu8x2.vv vd, vs1, vs2, vm # uint8×uint8→uint32 双步扩展MAC
vwmaccu8x2.vx vd, vs1, rs1, vm # uint8×scalar→uint32 双步扩展MAC
# 语义：vd[i] += vs1[i] * vs2[i] (double-widening MAC: uint8→uint32)
```

为何**RVV**现有**vwmaccu**不够：

RVV已提供`vwmaccu`指令，但仅支持单步扩展（uint8×uint8uint16累加），无法直接完成uint8uint32的双步扩展：

| RVV指令 | 操作 | 输入类型 | 输出类型 | 局限性 |
|---|---|---|---|---|
| `vwmaccu.vv` | widening MAC | uint8×uint8 | uint16累加（SEW=8时） | 单步扩展，仅输出uint16精度 |
| `vwmulu` + `vwaddu` | 分步widening | uint8uint16uint32 | uint32 | 需2条指令 |

现有`vwmaccu`的问题：

- 输入uint8元素相乘产生uint16中间结果
- 累加到uint16累加器（vd需为SEW'=16），无法直接得到uint32精度
- 无法直接完成uint8×uint8uint32的完整精度MAC

因此需要双步扩展**MAC**（double-widening MAC）指令`vwmaccu8x2`：

- 单指令完成uint8×uint8uint32完整精度乘累加
- 避免中间uint16阶段的精度截断

应用场景：

- MlasQgemmKernel K循环核心MAC操作
- INT8量化推理的所有GEMM/Conv层
- 当前RVV需vwmulu + vwaddu两条指令

性能对比：

| 方案 | 指令数 | 说明 |
|---|---|---|
| 当前RVV | 2 | vwmulu_vx + vwaddu_wv |
| 扩展后 | 1 | vwmaccu8x2.vx |
| 减少 | **50%** | K循环指令数减半 |

参考：x86 AVX512_VNNI `vpdpbusd`，ARM NEON `vudot.u8`

## [P1] vmatmul.rs.vv

指令定义：

```
vmatmul.rs.vv vd, vs1, vs2, vm # 2×8 × 8×2 矩阵块乘法
# vs1: 16个INT8元素（2行×8列矩阵A）
# vs2: 16个INT8元素（8行×2列矩阵B）
# vd: 4个INT32元素（2×2输出累加）
# 单指令完成32个MAC操作
```

应用场景：

- INT8矩阵乘法block-level优化
- 参考ARM SMMLA的2×2输出块处理模式

性能对比：

| 方案 | **32 MACs**指令数 | 说明 |
|---|---|---|
| 当前RVV | 4 | 2对vwmulu+vwaddu（每对处理16 MACs） |
| 扩展后 | 1 | vmatmul.rs.vv |
| 减少 | **75%** | 矩阵块指令数减少 |

**RVV**指令计数详解：

- SMMLA单指令：2×8 × 8×2 = 32 MACs（产生2×2=4个int32输出）
- RVV等效实现：需要4条指令（2对vwmulu+vwaddu）
- 每对vwmulu+vwaddu处理8个元素×2次乘加 = 16 MACs
- 需要两对处理32 MACs
- 注意：这是针对VLEN=128的情况（与SMMLA对比）
- 对于VLEN=512（16个int32元素），SMMLA等效工作需4条SMMLA（4×32=128 MACs），RVV需16条指令

硬件考量：

- 需16×int8乘法器阵列 + 4×int32加法树
- 预估面积约2×现有MAC单元
- 建议由硬件团队评估实现可行性

## [P2] vwmaccsu8x2.vv

指令定义：

```
vwmaccsu8x2.vv vd, vs1, vs2 # int8×uint8→int32 双步扩展MAC
# 内置符号处理，无需XOR 0x80
```

应用场景：

- U8×S8混合符号矩阵乘法
- ONNX Runtime INT8量化常用模式

性能对比：

- 当前：需手动XOR 0x80符号翻转 + vwmulu + vwaddu
- 扩展后：单指令内置符号处理

## [P3] vwmaccu8x2s.vv

指令定义：

```
vwmaccu8x2s.vv vd, vs1, vs2 # 饱和累加版本
# 累加值超出int32范围时饱和
```

应用场景：

- 大累加值场景避免溢出检测开销
- 参考：x86 VNNI `vpdpbusds`饱和版本

# 结论

## 关键发现

1. **RVV INT8 MAC**效率落后主流平台
- 相比AVX512_VNNI：指令数多50%（2条 vs 1条）
- 相比ARM SMMLA：指令数多4倍（4条 vs 1条，针对32 MACs）
- 相比Power MMA：指令数多16倍（16条 vs 1条，针对64 MACs）

2. 核心差距：缺乏双步扩展**MAC**指令
- RVV现有`vwmaccu`仅支持单步扩展（uint8uint16）
- x86 VNNI：`vpdpbusd`单指令uint8×int8int32
- ARM UDOT：`vudot.u8`单指令uint8×uint8uint32
- ARM SMMLA：单指令32×MAC（矩阵块）
- RVV：需2条指令完成uint8×uint8uint32完整精度MAC

3. 建议优先级
- **P0**：`vwmaccu8x2.vv` — 双步扩展MAC，解决所有INT8 MAC场景（-50%指令）
- **P1**：`vmatmul.rs.vv` — 矩阵块乘法，大型矩阵乘场景（-75%指令）
- **P2/P3**：符号混合、饱和版本 — 特定场景优化

## 整体收益估算

无BBV profiling数据，仅基于理论分析：

- K循环占比约85%（GEMM热点）
- P0 vwmaccu8x2：K循环指令数减半 整体估算减少42-50%
- 实际收益需BBV数据验证

## 优先级总结

| 优先级 | 扩展 | 收益范围 | 来源平台 |
|---|---|---|---|
| P0 | vwmaccu8x2.vv | K循环-50% | x86 VNNI / ARM UDOT |
| P1 | vmatmul.rs.vv | 矩阵块-75% | ARM SMMLA |
| P2 | vwmaccsu8x2.vv | 符号混合简化 | x86 VNNI |
| P3 | vwmaccu8x2s.vv | 饱和累加 | x86 VNNI |

# 审查日志

| 轮次 | 发现问题数 | 已修复 | 剩余 |
|---|---|---|---|
| R1 | - | - | 待审查 |
| R2 | 10 | 10 | 0 |
| R3 | 8 | 8 | 0 |

**R2**修复内容：

1. P0指令重命名为`vwmaccu8x2.vv`，明确双步扩展vs现有单步扩展区别
2. 修正50%收益计算依据：基于双步扩展MAC，非现有vwmaccu局限
3. Power10指令名修正：`xxmaccu8XVI8GER4PP`
4. SMMLA效率对比修正：32 MACs场景下4×差距（非64×）
5. VNNI指令描述修正：`vpdpbusd`为uint8×int8（非uint8×uint8）
6. 指令表修正：使用`vpdpbusd`非饱和版本
7. WASM指令名修正：`i16x8.extend_i32x4i32x4.dot_i16x8_s`
8. LSX/RVV差异说明补充：偶/奇分离vs乘/加分离策略对比
9. P1收益修正：75%（非95%），基于正确指令计数
10. 全文指令命名统一更新

**R3**修复内容：

1. Line 69 VNNI概述修正：`4×(uint8×uint8)4×(uint8×int8)`
2. Line 71 移除不存在指令：删除`vpdpbuud (U×U)`，保留4条有效VNNI指令
3. Line 80 移除不存在指令：删除`vpdpbsud (S×U)`，补充`vpdpbssds (S×S饱和)`
4. Line 257 WASM指令修正：删除不存在指令`i8x16.dot_i16x8_s`（核心特点与高价值指令表）
5. Line 222 Power MMA效率修正：`4× RVV16× RVV`（1 vs 16指令）
6. Line 77 vwmulu符号处理说明：添加uint8×int8需XOR 0x80处理注释
7. Line 299 vwmaccu输出类型修正：`uint16累加到uint32uint16累加（SEW=8时）`
8. Line 403 结论一致性修正：Power MMA差距改为16×（与表格一致）

最终审查结论：所有问题已修复。报告内容准确，VNNI指令名称与实际ISA一致（仅保留vpdpbusd/vpdpbusds/vpdpbssd/vpdpbssds）；SMMLA效率差距正确计算（4×）；Power MMA效率差距正确（16×）；vwmaccu8x2双步扩展指令语义明确；WASM SIMD指令名称正确。